

Engenharia de Software

Introdução

Profa. Rebeca Schroeder Freitas

rebeca.schroeder@udesc.br

1

Conferência da OTAN (Alemanha, 1968)

- 1a vez que o termo Engenharia de Software foi usado



Working Conference on Software Engineering

2

Comentário de participante da Conferência

"Certos sistemas estão colocando demandas que estão além das nossas capacidades... **Estamos tendo dificuldades com grandes aplicações.**"

3

Definição de Engenharia de Software

Área da Computação destinada a investigar os desafios e propor soluções que permitam desenvolver sistemas de software — principalmente aqueles mais complexos e de maior tamanho — de forma produtiva e com qualidade

4

Dificuldades Essenciais

Complexidade
Conformidade
Facilidade de Mudanças
Invisibilidade



Tornam Engenharia de Software diferente de outras engenharias

5

O que se estuda em ES?

1. Engenharia de Requisitos
2. Projeto de Software
3. Construção de Software
4. Testes de Software
5. Manutenção de Software
6. Gerência de Configuração
7. Gerência de Projetos



6

O que se estuda em ES? (cont.)



- 8. Processos de Software
- 9. Modelos de Software
- 10. Qualidade de Software
- 11. Prática Profissional
- 12. Aspectos Econômicos

(1) Requisitos de Software

- Requisitos: **o que** o sistema deve fazer e **como** para atender aos seus clientes com qualidade de serviço
- Definir, analisar, documentar e validar os requisitos dos sistemas

Requisitos Funcionais vs Não-Funcionais

- **Funcionais:** "o que" um sistema deve fazer
 - Funcionalidades ou serviços ele deve implementar
 - Exemplos para um sistema de *home-baking*:
 - Informar o saldo da conta, realizar transferências, ...
- **Não-funcionais:** "como" um sistema deve operar
 - Sob quais restrições e com qual qualidade de serviço

Exemplos de Requisitos Não-Funcionais

- Desempenho: dar o saldo da conta em 5 segundos
- Disponibilidade: estar no ar 99.99% do tempo
- Capacidade: armazenar dados de 1M de clientes
- Tolerância a falhas: continuar operando se São Paulo cair
- Segurança: criptografar dados trocados com as agências

Exemplos de Requisitos Não-Funcionais (cont.)

- Privacidade: não armazenar localização dos usuários
- Interoperabilidade: se integrar com os sistema do BACEN
- Manutenibilidade: bugs devem ser corrigidos em 24 hs
- Usabilidade: versão para celulares e tablets

(2) Projeto de Software

- Definição dos principais **unidades de código** de um sistema
- Apenas no nível de **interfaces** (providas e requeridas)

```
class ContaBancaria {
    private Cliente cliente;
    private double saldo;
    public double getSaldo() { ... }
    public String getNomeCliente() { ... }
    public String getExtrato (Date inicio) { ... }
    ...
}
```

(2) Projeto de Software (cont.)

- Quando o projeto é realizado em um nível de abstração maior do que de classes, por exemplo no nível de **pacotes**, é chamado de **projeto da arquitetura do Software**.

13

(3) Construção de Software

- Implementação (codificação) do sistema
- Algumas preocupações:
 - Algoritmos e estruturas de dados
 - Bibliotecas e frameworks de terceiros
 - Ferramentas: IDEs, depuradores, etc
 - Padrões de nome, documentação

14

(4) Testes de Software

- Verificam se um programa apresenta um resultado esperado ao ser executado com casos de teste
- Podem ser:
 - Manuais
 - Automatizados

15

Frase famosa

- Testes de software mostram a presença de bugs, mas não a sua ausência. -- Edsger W. Dijkstra



16

Outra observação importante



Allen Holub
@allenholub

All software is tested. The real question is whether the tester is you or your users.

12:49 PM · May 28, 2020 · TweetDeck

17

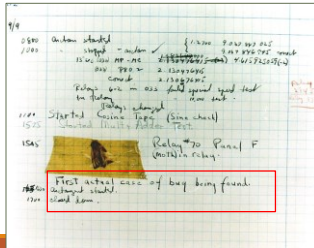
Defeitos, Bugs, Fallhas

- O seguinte código possui um **defeito** ou um **bug**:


```
if (condition)
    area = pi * raio * raio * raio;    // código defeituoso;
```
- O certo é "área é igual a pi vezes raio ao quadrado"
- Quando esse código for executado ele vai causar uma **falha** (*failure*); ou seja, um resultado errado
- Segundo a IEEE, **falta** (*fault*) ou **erro** é o mesmo que defeito

18

Um dos primeiros bugs (diário de Grace Hopper, 1947)



19

Falha Famosa: Explosão do Ariane 5 (1996)



20

30 segundos depois

Custo do foguete e satélite:
US\$ 500 milhões



Photo of Ariane 501 Flight, a few seconds after explosion (Credits ESA 1996)

21

Relatório do Comitê de Investigação

- Explosão foi causada por uma falha de software
- Conversão de um real de 64 bits para um inteiro de 16 bits
- Como o real não "cabia" em 16 bits, a conversão falhou

22

Verificação vs Validação

- **Verificação:** estamos implementando o sistema corretamente?
 - De acordo com os requisitos e especificações
- **Validação:** estamos implementando o sistema correto?
 - O sistema que os clientes querem

23

(5) Manutenção de Software

- Tipos de Manutenção:
 - Corretiva
 - Preventiva
 - Adaptativa
 - Refactoring
 - Evolutiva

24

Exemplo de Manutenção Preventiva: Y2K Bug

- Manutenção realizada no "tamanho" de campos *data*, de DD-MM-AA para DD-MM-AAAA
- Riscos e prejuízos foram super-estimados



25

Refactoring

- Manutenção para melhorar a manutenibilidade; ou seja, não corrige bugs, não implementa novas funcionalidades
- Exemplos:
 - Renomear variável, classe, etc
 - Extrair função
 - Mover função
 - etc

26

Evolutiva – ex. Sistemas Legados

- Sistemas antigos, usando linguagens, SOs, BDs antigos
- Manutenção custosa e arriscada
- Muitas vezes, são muito importantes (legado ≠ irrelevante)

27

COBOL ainda é muito comum em bancos

- Estima-se que existam ~200 bilhões de LOC em COBOL
- Maioria são sistemas de bancos
 - 95% das transações em ATMs são em COBOL
 - Um único banco europeu tem 250 MLOC em COBOL

Fonte: palestra de Vadim Zaytsev na SLE 2020 (<https://youtu.be/sSkUTdIDjs>)

28

(6) Gerência de Configuração

- Todo software é desenvolvido usando um sistema de controle de versões (exemplo: git)
- Atua como uma "fonte da verdade" sobre o código
- Permite recuperar versões antigas

29

(6) Gerência de Configuração (cont.)

- Versionamento semântico (release x.y.z):
 - X: versão com adição de novas funcionalidades muito diferentes da versão anterior (*major version*)
 - Y: versão com adição de novas mas pequenas funcionalidades (*minor version*)
 - Z: versão com apenas correções de *bugs* (normalmente chamada de *patch*)

30

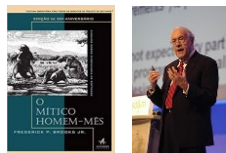
(7) Gerência de Projetos

- Algumas atividades:
 - Negociação contratos (prazos, valores, cronograma)
 - Gerência de RH (contratação, treinamento etc)
 - Gerenciamento de riscos
 - Acompanhamento da concorrência, marketing, etc

31

Lei de Brooks

- Incluir novos devs em um projeto que está atrasado, vai deixá-lo mais atrasado ainda



32

Stakeholders

- Todas as "partes interessadas" em um projeto de software
- São aqueles que **afetam** ou **são afetados** pelo projeto
- Isto é, desenvolvedores, clientes, usuários, gerentes, empresas terceirizadas, fornecedores, governo etc

33

(8) Processos de Desenvolvimento de Software

- Processo de software: define as atividades e etapas que devem ser seguidas para construir um sistema de software
- Dois principais modelos:
 - Waterfall ("cascata")
 - Ágil

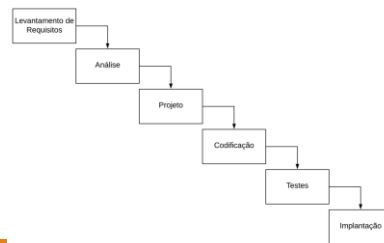
34

Modelo em Cascata

- Inspirado em processos de engenharias tradicionais, como Civil, Mecânica, Elétrica, etc
- Proposto na década de 70 e muito usado até ~1990
 - Exigência do uso deste modelo por padronização do Dep. De Defesa norte-americano em 1985.

35

Modelo em Cascata



36

Problemas com Modelo Waterfall

- Requisitos mudam com frequência
 - Levantamento completo de requisitos demanda tempo
 - Quando ficar pronto, o "mundo já mudou"
 - Clientes às vezes não sabem o que querem
- Documentações de software são extensas
- E rapidamente se tornam obsoletas

37

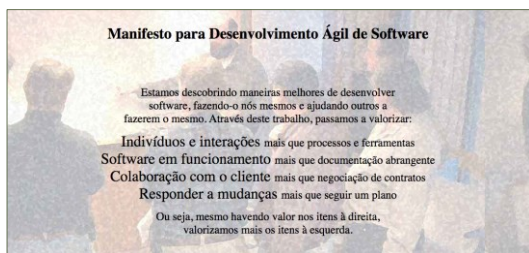
Manifesto Ágil (2001)

- Encontro de 17 engenheiros de software em Utah
- Crítica a modelos sequenciais e pesados
- Novo modelo: incremental e iterativo



<https://iamchannarkit.co.th/history-some-pictures-and-pdfs-of-the-agile-manifesto-meeting-on-2001-a33o40bc2b>

38



<https://agilemanifesto.org/iso/ptbr/manifesto.html>

39

Desenvolvimento Ágil

- Profundo impacto na indústria de software
- Hoje, tudo é ágil... Talvez adjetivo até desgastado

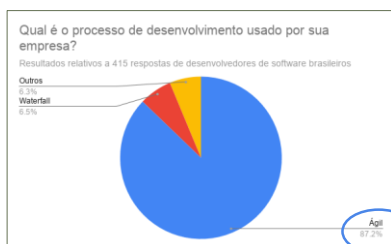


Março 2019

Maio 2020

40

A seguinte pergunta foi feita para 415 devs brasileiros



Fonte: Surveying the Impacts of COVID-19 on the Perceived Productivity of Brazilian Software Developers. SBES 2020

41

(9) Modelagem de Software

- Modelo: representação mais alto nível do que o código
- Usado para:
 - explicar o "design" para os devs
 - documentar o projeto
 - facilitar compreensão e manutenção
- Modelos como *sketches* (esboço)

42

Unified Modeling Language (UML)

- Linguagem gráfica para modelagem de software

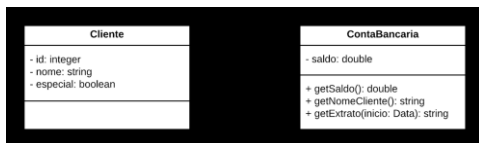


Diagrama de Classes UML

(10) Qualidade de Software

- Qualidade Externa:
 - Correção, robustez, extensibilidade, reusabilidade, eficiência, compatibilidade, facilidade de uso, portabilidade
- Qualidade Interna:
 - Modularidade, legibilidade, testabilidade, manutenibilidade, etc

(10) Qualidade de Software (cont.)

- Estratégias para garantia da qualidade de software:
 - Métricas:
 - LOCs (métrica de produto - tamanho)
 - # de defeitos por LOC, ... (métrica de processo)
 - Revisões de código:
 - Detectar defeitos
 - Garantir qualidade interna

(11) Prática Profissional

- Ética
- Certificações
- Regulamentação da Profissão
- Cursos de graduação
- Currículo de Referência

Aspectos Éticos

- Engenheiros de Software começam a questionar o uso que as empresas fazem do software desenvolvido por eles

Cybersecurity
Google Engineers Refused to Build Security Tool to Win Military Contracts
 A work boycott from the Group of Nine is yet another hurdle to the company's efforts to compete for sensitive government work.

<https://www.bloomberg.com/news/articles/2018-05-21/google-engineers-refused-to-build-security-tool-to-win-military-contracts>

(12) Aspectos Econômicos

- Qual o retorno de investimento?
- Como monetizar meu software? (anúncios, assinaturas, etc)
- Quanto cobrar pelo desenvolvimento de um sistema?
- Qual a melhor licença para o meu software?

Para finalizar: Tipos ABC de sistemas

- Proposta por Bertrand Meyer
- Três tipos de software:
 - Sistemas C (Casuais)
 - Sistemas B (Business)
 - Sistemas A (Acute)

40

Sistemas C (Casuais)

- Tipo muito comum de sistema
- Sistemas pequenos, sem muita importância
- Podem ter bugs; às vezes, são descartáveis
- Desenvolvidos por 1-2 engenheiros
- Risco: "over-engineering"

41

Sistemas B (Business)

- Sistemas importantes para uma organização
- **Sistemas beneficiam-se do que veremos nesta disciplina**
- Risco: não usarem técnicas de ES e se tornarem um passivo, em vez de um ativo para as organizações

51

Sistemas A (Acute)

- Sistemas onde nada pode dar errado, pois o custo é imenso, em termos de vidas humanas e/ou \$\$\$
- Também chamados sistemas de missão crítica



Metrô



Aviação



Medicina

52

Sistemas A (Acute)

- Requerem certificações
- **Fora do escopo desta disciplina**

Document Title	
DO-178C - Software Considerations in Airborne Systems and Equipment Certification	
Description	
This document provides recommendations for the production of software for airborne systems and equipment that performs its intended function with a level of confidence in safety that complies with airworthiness requirements. Compliance with the objectives of DO-178C is the primary means of obtaining approval of software used in civil aviation products.	
Document Number	DO-178C
Format	Hard Copy
Committee	SC-285
Issue Date	12/13/2011

53

Referências

- PRESSMAN, R. S.. Engenharia de Software. Makron Books. 1995.
- Valente, M. T. Engenharia de Software Moderna. Editora Independente, 2020 (<https://engsoftmoderna.info/>)

Fim

